

AI FOR SMART AGRICULTURE ADMINISTRATION

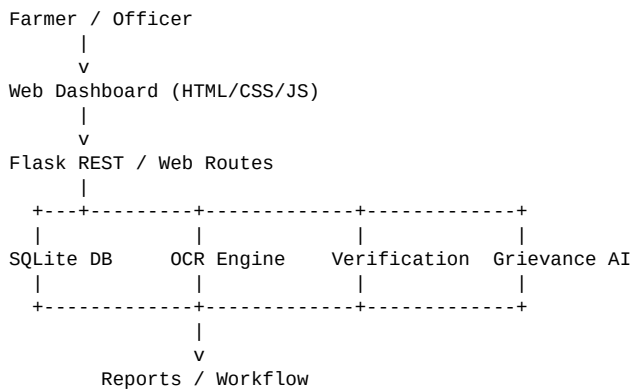
Prototype Design & Source Code Documentation

Project objective: Build an AI-assisted agricultural office system that digitizes farmer applications, performs document/OCR-based verification, detects inconsistencies, routes cases through workflows, prioritizes grievances, and generates administrative reports.

Module	Prototype Function
Farmer Records	Register/search farmer and application information
Document Processing	Extract text from uploaded documents using OCR
Verification	Check required fields, document completeness and data consistency
AI Risk/Anomaly	Flag suspicious or inconsistent records for official review
Workflow	Track submitted → verification → officer review → approved/rejected
Grievance Management	Classify and prioritize complaints
Reports	Show application counts, status and pending workload

1. Prototype Architecture

A lightweight web prototype is proposed using Python Flask, SQLite, HTML/CSS/JavaScript and optional OCR/ML components. The prototype is intentionally simple enough for a college demonstration while leaving clear extension points for production AI services.



2. Suggested Project Folder

```
smart-agri-admin/
├──
├── app.py
├── database.py
├── ai_engine.py
├── requirements.txt
├── agriculture.db
├── templates/
│   ├── dashboard.html
├── static/
│   ├── style.css
├── uploads/
```

3. requirements.txt

```
Flask
pandas
scikit-learn
pytesseract
Pillow
```

4. Main Flask Prototype — app.py

```
from flask import Flask, render_template, request, jsonify
import sqlite3
from datetime import datetime
from ai_engine import verify_application, classify_grievance

app = Flask(__name__)

def get_db():
    conn = sqlite3.connect("agriculture.db")
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db()
    conn.execute("""CREATE TABLE IF NOT EXISTS applications(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        farmer_name TEXT NOT NULL,
        aadhaar_last4 TEXT,
        scheme TEXT,
        income REAL,
        land_acres REAL,
        status TEXT DEFAULT 'Submitted',
        risk_score REAL DEFAULT 0,
        created_at TEXT
    )""")
    conn.execute("""CREATE TABLE IF NOT EXISTS grievances(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        farmer_name TEXT,
        complaint TEXT,
        category TEXT,
        priority TEXT,
        status TEXT DEFAULT 'Open',
        created_at TEXT
    )""")
    conn.commit()
    conn.close()

@app.route("/")
def dashboard():
    conn = get_db()
    applications = conn.execute(
        "SELECT * FROM applications ORDER BY id DESC"
    ).fetchall()
    grievances = conn.execute(
        "SELECT * FROM grievances ORDER BY id DESC"
    ).fetchall()
    conn.close()
    return render_template("dashboard.html",
        applications=applications,
        grievances=grievances)

@app.route("/application", methods=["POST"])
def add_application():
    data = request.get_json()
    result = verify_application(data)

    conn = get_db()
    conn.execute("""INSERT INTO applications
        (farmer_name, aadhaar_last4, scheme, income, land_acres,
        status, risk_score, created_at)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)""",
        (data.get("farmer_name"), data.get("aadhaar_last4"),
        data.get("scheme"), data.get("income", 0),
        data.get("land_acres", 0), result["status"],
        result["risk_score"], datetime.now().isoformat()))
    conn.commit()
    conn.close()

    return jsonify(result)

@app.route("/grievance", methods=["POST"])
def add_grievance():
    data = request.get_json()
    result = classify_grievance(data.get("complaint", ""))

    conn = get_db()
    conn.execute("""INSERT INTO grievances
        (farmer_name, complaint, category, priority, created_at)
    VALUES (?, ?, ?, ?, ?)""",
        (data.get("farmer_name"), data.get("complaint"),
```

```

        result["category"], result["priority"],
        datetime.now().isoformat())
    conn.commit()
    conn.close()

    return jsonify(result)

if __name__ == "__main__":
    init_db()
    app.run(debug=True)

```

5. AI / Verification Engine — ai_engine.py

```

def verify_application(data):
    risk = 0
    reasons = []

    required = ["farmer_name", "scheme", "income", "land_acres"]

    for field in required:
        if not data.get(field):
            risk += 25
            reasons.append(field + " missing")

    try:
        income = float(data.get("income", 0))
        land = float(data.get("land_acres", 0))

        if income < 0 or land < 0:
            risk += 40
            reasons.append("Invalid numeric value")

        # Simple anomaly rule for prototype demonstration
        if land > 100:
            risk += 20
            reasons.append("Large land-area value requires review")

    except ValueError:
        risk += 40
        reasons.append("Invalid number format")

    if risk >= 50:
        status = "Officer Review"
    else:
        status = "Verified"

    return {
        "status": status,
        "risk_score": min(risk, 100),
        "reasons": reasons
    }

def classify_grievance(text):
    t = text.lower()

    if any(word in t for word in ["insurance", "claim", "crop loss"]):
        category = "Insurance"
    elif any(word in t for word in ["subsidy", "payment", "amount"]):
        category = "Subsidy"
    elif any(word in t for word in ["scheme", "application"]):
        category = "Scheme Application"
    else:
        category = "General"

    if any(word in t for word in ["urgent", "not received", "failed",
                                   "loss", "emergency"]):
        priority = "High"
    elif len(t) > 120:
        priority = "Medium"
    else:
        priority = "Normal"

    return {"category": category, "priority": priority}

```

6. Dashboard — templates/dashboard.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Smart Agriculture Administration</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
<header>
  <h1>AI Smart Agriculture Administration</h1>
  <p>Officer Dashboard</p>
</header>

<main>
<section class="cards">
  <div class="card">
    <h3>Applications</h3>
    <p>{{ applications|length }}</p>
  </div>
  <div class="card">
    <h3>Grievances</h3>
    <p>{{ grievances|length }}</p>
  </div>
</section>

<section class="panel">
<h2>Recent Applications</h2>
<table>
<tr><th>ID</th><th>Farmer</th><th>Scheme</th>
<th>Status</th><th>Risk</th></tr>
{% for a in applications %}
<tr>
<td>{{ a.id }}</td>
<td>{{ a.farmer_name }}</td>
<td>{{ a.scheme }}</td>
<td>{{ a.status }}</td>
<td>{{ a.risk_score }}</td>
</tr>
{% endfor %}
</table>
</section>

<section class="panel">
<h2>Grievance Queue</h2>
<table>
<tr><th>Farmer</th><th>Complaint</th>
<th>Category</th><th>Priority</th></tr>
{% for g in grievances %}
<tr>
<td>{{ g.farmer_name }}</td>
<td>{{ g.complaint }}</td>
<td>{{ g.category }}</td>
<td>{{ g.priority }}</td>
</tr>
{% endfor %}
</table>
</section>
</main>
</body>
</html>
```

7. Basic Styling — static/style.css

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f4f6f8;
}

header {
  padding: 24px;
  background: #1f2937;
  color: white;
}

main {
  padding: 24px;
}

.cards {
```

```
        display: flex;
        gap: 20px;
    }

    .card, .panel {
        background: white;
        padding: 18px;
        margin-bottom: 20px;
        border-radius: 10px;
        box-shadow: 0 2px 8px #ddd;
    }

    .card {
        min-width: 180px;
    }

    table {
        width: 100%;
        border-collapse: collapse;
    }

    th, td {
        padding: 10px;
        border-bottom: 1px solid #ddd;
        text-align: left;
    }
}
```

8. Optional OCR Module

For a demonstration, an uploaded farmer document can be passed through Tesseract OCR. The extracted text should be treated as a verification aid; final approval remains with the authorized officer.

```
import pytesseract
from PIL import Image

def extract_text(image_path):
    image = Image.open(image_path)
    text = pytesseract.image_to_string(image)
    return text

# Example:
# text = extract_text("uploads/document.jpg")
# print(text)
```

9. Example API Request

```
POST /application
Content-Type: application/json

{
  "farmer_name": "Ravi Kumar",
  "aadhaar_last4": "1234",
  "scheme": "Crop Subsidy",
  "income": 85000,
  "land_acres": 3.5
}
```

10. Expected Prototype Flow

1. Farmer/application data is entered.
2. System validates mandatory fields.
3. OCR extracts text from supporting documents when uploaded.
4. Verification engine checks consistency and flags anomalies.
5. Application is marked Verified or sent to Officer Review.
6. Grievances are automatically categorized and prioritized.
7. Officer dashboard displays pending workload and status.
8. Reports can be generated from the SQLite records.

11. Future AI Extensions

- Train an anomaly-detection model using historical administrative data.
- Use document classifiers to identify certificates, receipts and claim forms.
- Add semantic search over farmer records and government scheme documents.
- Add duplicate-record and tamper-evidence detection.
- Add multilingual grievance classification for Tamil and other regional languages.
- Add role-based access, audit logs, encryption, backups and official authentication before real deployment.

12. Important Prototype Note

This is a **college demonstration prototype**, not a production agricultural-government system. AI-generated risk scores, OCR results and classifications should assist—not automatically replace—authorized human verification. Sensitive farmer information should be minimized and protected.